

SENIOR SWE INTERVIEW SERIES

Module 1: Networking Basics

Interview-focused networking, as asked at Google, Meta, Amazon, Microsoft, Uber, Netflix, Cloudflare, Stripe, LinkedIn & Cisco

Part of: Computer Networking for Senior Software Engineer Interviews

Built July 02, 2026

MODULE 1 — Networking Basics (Interview Perspective)

Audience: Backend / distributed-systems engineers with 5+ years of experience. Goal: Answer OSI/TCP-IP/MTU questions the way a Principal Engineer at Google, Meta, or Cloudflare would — with packet-level precision and production war stories, not textbook recitals.

Topic 1.1 — OSI Model (Interview Perspective)

1. Why Interviewers Ask This

Interviewers almost never want the seven layers memorized. They use the OSI model as a *vocabulary check*: when you say “that’s an L4 problem, not an L7 problem” during a system design or debugging round, you demonstrate you can localize failures. At Meta and Cloudflare, “Which layer would you debug first?” is a classic screen for on-call maturity.

2. Core Concept

The OSI model is a 7-layer separation of concerns. In practice, engineers care about four of them:

Layer	Name	What backend engineers touch
L7	Application	HTTP, gRPC, DNS, TLS payloads
L4	Transport	TCP/UDP ports, retransmission, congestion
L3	Network	IP routing, ICMP, MTU
L2	Data Link	Ethernet, ARP, switch behavior

L5 (Session) and L6 (Presentation) are absorbed into L7 in the real world (TLS straddles L5–L7 depending on who you ask — a great nuance to mention in interviews).

3. Internal Working

Each layer adds its own header and treats everything above it as opaque payload. A router only parses up to L3; a switch only up to L2; a load balancer parses up to L4 or L7 depending on its type. This is *exactly* why an L4 LB is faster than an L7 LB — less parsing.

4. Packet Flow Explanation

When your service calls `GET /api/users`: 1. App writes HTTP bytes → kernel socket buffer (L7). 2. TCP segments the stream, adds ports + seq numbers (L4). 3. IP adds src/dst addresses, TTL (L3). 4. Ethernet adds MAC addresses + FCS (L2). 5. NIC serializes bits onto the wire (L1). Every intermediate hop strips L2, inspects L3 for routing, re-wraps L2 for the next hop. L4+ headers are untouched end-to-end (unless NAT/middleboxes interfere — mention this, it’s a senior signal).

5. ASCII Diagram

```

App data:           [ HTTP request           ]
L4 (TCP):           [ TCP hdr|HTTP request    ]
L3 (IP):            [ IP hdr|TCP hdr|HTTP request ]
L2 (Eth): [ Eth|IP hdr|TCP hdr|HTTP request | FCS ]
               |
               v bits on wire (L1)
Router hop: strips Eth -> reads IP dst -> new Eth -> forwards
(TCP + HTTP untouched end-to-end)

```

6. Real Production Example

Cloudflare's edge: Magic Transit operates at L3 (IP), Spectrum at L4 (TCP/UDP proxy), and the CDN/WAF at L7 (HTTP). One company, three products, one per layer — this is the cleanest real-world proof that the layer model drives product architecture.

7. Advantages

- Fault isolation: “ping works, curl fails” instantly rules out L1–L3.
- Independent evolution: HTTP/3 changed L4 semantics (QUIC over UDP) without touching L3.
- Clear team ownership boundaries (network team owns L1–L3, service teams own L7).

8. Trade-offs

- Layering hides information: TCP can't know a WiFi link dropped a frame vs. congestion — it assumes congestion and slows down (wrong on lossy wireless).
- Encapsulation overhead: each header steals payload bytes (see MTU/MSS below).
- Middleboxes violate layering (NAT rewrites L3+L4; L7 firewalls parse everything), causing “works on my machine” bugs.

9. Common Mistakes

- Reciting all 7 layers when asked “explain OSI” — interviewers want the *practical* 4.
- Saying “TLS is layer 6” as a fact; it's contested. Say “TLS sits between L4 and L7 in practice.”
- Claiming routers look at TCP ports (they don't, unless doing policy routing/NAT).

10. Performance Impact

Layer at which you process a packet dictates cost: L3 routing is ASIC-speed (Tbps), L4 LB is ~millions of pps per core, L7 proxying is ~tens of thousands of RPS per core (parsing, buffering, TLS). This single fact drives most load-balancer architecture questions.

11. Common Interview Questions

1. “curl to a service times out but ping works — walk me through layers.”
2. “Why is an L4 load balancer faster than L7?”
3. “At which layer does a switch vs. router vs. LB operate?”
4. “Where does TLS fit in the OSI model?”

12. Follow-up Questions

- “What breaks layering in real networks?” → NAT, transparent proxies, DPI firewalls, TCP-terminating CDNs.
- “How does QUIC blur the layers?” → transport implemented in userspace over UDP, crypto fused into transport.

13. Debugging Scenarios

- **Ping OK, TCP connect fails** → L4 issue: firewall/security-group blocking the port, or no listener (`ss -ltnp`).
- **TCP connects, HTTP hangs** → L7 issue: app thread pool exhausted, slow upstream.
- **Intermittent resets only through one path** → middlebox (IDS/NAT timeout) at L3/L4.

14. Best Practices

- Debug bottom-up for connectivity (link → IP → port → app), top-down for latency (app metrics → TCP retransmits → link errors).
- In design interviews, always name the layer of every box you draw (“this LB is L4 passthrough, TLS terminates at L7 here”).

15. Practice Questions

1. A service behind an L4 LB sees client IPs as the LB’s IP. Why? What are 3 fixes? (Answer sketch: L4 SNAT; fixes = DSR, PROXY protocol, X-Forwarded-For at L7.)
2. Explain why a bigger L2 frame doesn’t help if L3 MTU is 1500.
3. Your gRPC calls fail through one datacenter path only. Which layers do you suspect and in what order?

Topic 1.2 — TCP/IP Model

1. Why Interviewers Ask This

Because it’s the model actually implemented in kernels. Interviewers at Amazon/Google check whether you know the mapping to real stack components (`socket()` API boundary, where the kernel/user split lives).

2. Core Concept

4 layers: **Application** (HTTP, DNS, gRPC — userspace), **Transport** (TCP/UDP — kernel), **Internet** (IP/ICMP — kernel), **Link** (Ethernet/WiFi — driver + NIC). The socket API is the boundary between your code and the kernel’s transport layer.

3. Internal Working

- `write()` on a socket copies bytes to the kernel **send buffer**; TCP decides *when* and *how much* to send (Nagle, congestion window).
- Receive path: NIC → ring buffer → softirq → IP → TCP → socket **receive buffer** → your `read()`.
- This is why “my write() returned” ≠ “the peer got it” — a classic interview trap.

4. Packet Flow Explanation

`send(fd, buf, n)` → data sits in send buffer → TCP cuts MSS-sized segments → IP encapsulation → NIC (often with **TSO**: the NIC itself does segmentation). Inbound, **GRO** coalesces segments before TCP sees them. Mentioning TSO/GRO signals real production familiarity.

5. ASCII Diagram

```

Userspace | your app --- write()/read() --- socket API
-----+-----
Kernel    | TCP/UDP (send/recv buffers, cwnd, rto)
          | IP      (routing table, fragmentation)
-----+-----
Driver/NIC | ring buffers, TSO/GRO, checksum offload
-----+-----
Wire       | Ethernet frames

```

6. Real Production Example

Netflix serves ~hundreds of Gbps per FreeBSD box using `sendfile()` + kernel TLS (kTLS): data goes disk → NIC without entering userspace. Understanding the layer split is what makes optimizations like this possible.

7. Advantages

- Kernel handles reliability/congestion once for all apps.
- Clean upgrade path: apps don't change when kernel improves congestion control (e.g., enabling BBR fleet-wide).

8. Trade-offs

- Kernel transport = syscall + copy overhead; that's why QUIC (userspace) and DPDK/XDP (kernel bypass) exist.
- Kernel upgrade cycles are slow — TCP evolves in years; QUIC iterates in weeks (Google's stated reason for QUIC).

9. Common Mistakes

- Confusing OSI-7 and TCP/IP-4 numbering mid-answer.
- Believing `write()` success means delivery. It only means "buffered in kernel."
- Forgetting UDP also lives in the transport layer (people say "transport = TCP").

10. Performance Impact

Each syscall $\approx 1\text{--}2\ \mu\text{s}$; each copy costs memory bandwidth. High-perf servers batch (`writerv`, `sendmmsg`), use zero-copy (`sendfile`), or bypass the kernel entirely. Knowing where copies happen is a senior-level differentiator.

11. Common Interview Questions

1. "What happens in the kernel when you call `write()` on a TCP socket?"
2. "Why did Google build QUIC in userspace?"

3. “What’s the difference between the send buffer being full and the network being slow?” (Trick: they’re the same signal to your app — write blocks/EAGAIN.)

12. Follow-up Questions

- “How would you detect the send buffer is the bottleneck?” → `ss -tmi`, watch `send-q`.
- “What is TSO and why can tcpdump show 64KB ‘packets’?” → capture happens before NIC segmentation.

13. Debugging Scenarios

- App “slow to send”: check `ss -tnpi` — if `send-q` large and cwnd small → network congestion; if `send-q` empty → app isn’t writing (app bug, not network).
- High CPU in `softirq` → packet rate problem, consider RSS/GRO tuning.

14. Best Practices

- Size `SO_SNDBUF/SO_RCVBUF` to $\text{bandwidth} \times \text{RTT}$ for high-BDP links (or rely on Linux autotuning; don’t clamp it accidentally with `setsockopt`).
- Use `ss` over `netstat` (`netstat` is deprecated and slow).

15. Practice Questions

1. Your service writes 1MB responses; clients on 200ms-RTT links see 3s transfers. Compute the buffer size needed. ($\text{BDP} = \text{rate} \times \text{RTT}$; if `buffer = 64KB` → $\text{max throughput } 64\text{KB}/200\text{ms} \approx 2.6 \text{ Mbps.}$)
2. Why does tcpdump on the sender show fewer, larger packets than on the receiver?

Topic 1.3 — Encapsulation

1. Why Interviewers Ask This

It’s the mechanism behind VPNs, service meshes, and overlay networks (VXLAN in Kubernetes/EC2). Cloud interviews (AWS, Google Cloud) love “how does a packet get from pod A to pod B on another node?”

2. Core Concept

Each layer prepends its header to the payload from above. Extended meaning: **tunneling** = putting a full packet inside another packet (IP-in-IP, VXLAN = Ethernet-in-UDP, GRE, WireGuard = IP-in-UDP+crypto).

3. Internal Working

Headers added on send: HTTP → +TCP(20–60B) → +IP(20B) → +Ethernet(14B+4B FCS). For overlays: the whole original frame becomes payload of a new UDP/IP packet with the *node*’s addresses, so the physical network only needs to route node-to-node.

4. Packet Flow Explanation

Kubernetes pod-to-pod across nodes (VXLAN): 1. Pod A sends packet with pod IPs (10.244.1.5 → 10.244.2.9). 2. Node’s VTEP wraps it: outer UDP/4789 + outer IP (nodeA → nodeB). 3. Physical network routes on node IPs only. 4. Node B unwraps, delivers original packet to pod B.

5. ASCII Diagram

```
Original:      [IP podA->podB [TCP [app data]]]
VXLAN wrap:
[Eth][IP nodeA->nodeB][UDP:4789][VXLAN][Eth][IP podA->podB][TCP[data]]
^----- outer (physical net) -----^ ^----- inner (overlay) -----^
```

6. Real Production Example

AWS VPC: every packet between EC2 instances is encapsulated by the Nitro card with a proprietary overlay header carrying VPC ID — this is how millions of VPCs share one physical network with overlapping IP ranges.

7. Advantages

- Overlay networks decouple logical addressing from physical topology.
- Layer independence: swap WiFi for Ethernet, nothing above L2 changes.

8. Trade-offs

- Every header layer shrinks usable payload (VXLAN eats ~50 bytes → effective MSS drops).
- Nested encapsulation makes MTU math fragile (the #1 cause of “large requests hang, small ones work”).
- Encap/decap costs CPU unless offloaded to the NIC.

9. Common Mistakes

- Forgetting to reduce MTU inside tunnels → silent blackholing of full-size packets.
- Thinking encapsulation encrypts (VXLAN/GRE are plaintext; only WireGuard/IPsec/TLS encrypt).

10. Performance Impact

50B overhead on 1450B payload ≈ 3.5% throughput loss — negligible. The real cost: losing NIC offloads (TSO/checksum) for encapsulated traffic can halve throughput; modern NICs offer VXLAN offload.

11. Common Interview Questions

1. “How do two pods on different nodes communicate?” (K8s round)
2. “Draw the headers of a VXLAN packet.”
3. “Why does my VPN break large file uploads?” (MTU trap)

12. Follow-up Questions

- “What MTU should you set inside a VXLAN overlay on a 1500-MTU network?” → 1450 (or enable jumbo frames outside).
- “How do overlapping IP ranges work in cloud VPCs?” → tenant ID in the encapsulation header.

13. Debugging Scenarios

- SSH works, `scp` of a big file hangs over VPN → tunnel MTU; test with `ping -M do -s 1400`.
- Cross-node pod traffic fails, same-node works → UDP/4789 blocked between nodes.

14. Best Practices

- Always account for tunnel overhead in MTU planning; enable jumbo frames (9000) inside datacenters.
- Use MSS clamping on tunnel endpoints as a safety net.

15. Practice Questions

1. Physical MTU 1500, VXLAN (50B) + WireGuard (60B) nested. What's the max inner TCP payload? (1500-50-60-20 IP-20 TCP = 1350.)
2. Why can a switch forward VXLAN traffic without understanding VXLAN?

Topic 1.4 — Decapsulation

1. Why Interviewers Ask This

Receive-path understanding separates people who've debugged production from those who've read blogs — especially “where can a packet be dropped on the way up the stack?”

2. Core Concept

The reverse of encapsulation: each layer strips its header, validates it, and hands the payload up. Every validation step is a potential silent drop point.

3. Internal Working

NIC checks Ethernet FCS (bad frame → dropped, counter `rx_crc_errors`). Kernel IP layer checks checksum, TTL, destination. TCP checks checksum, ports (no listener → RST), sequence validity (out-of-window → dropped/ACKed). Finally data lands in the socket receive buffer — if it's full, TCP advertises window 0 rather than dropping.

4. Packet Flow Explanation

Frame arrives → DMA into ring buffer → IRQ/NAPI poll → GRO merge → IP validation → route decision (local? forward?) → TCP demux by 4-tuple → sequence check → receive buffer → app `read()`. Drops possible at: ring overflow, conntrack table full, socket backlog full, receive buffer full.

5. ASCII Diagram

```
wire -> [NIC: FCS check] -> ring buffer -> [IP: cksum/TTL/dst]
      -> [TCP: port? seq? cksum?] -> recv buffer -> app read()
Drop points: FCS fail | ring full | no route | no listener(RST)
              | out-of-window | buffer full (win=0, not a drop)
```

6. Real Production Example

A classic outage pattern at scale (seen at GitHub, Shopify postmortems): SYN floods or connection storms fill the **listen backlog**; the kernel silently drops SYNs; clients see timeouts while the server looks “idle.” Fix: raise `somaxconn`/backlog + SYN cookies.

7. Advantages

- Validation at each layer keeps corrupt data from reaching apps.
- Demultiplexing by port lets thousands of services share one IP.

8. Trade-offs

- Silent drops at multiple layers make debugging hard — you must know the counters (`netstat -s` , `ethtool -S` , `nstat`).
- Checksum offload means corruption *can* sneak through broken NIC hardware (rare but real; famous 2015-era “bit flip” incidents).

9. Common Mistakes

- Assuming “no error in app logs = packet arrived.” Kernels drop silently.
- Ignoring `ListenOverflows` / `ListenDrops` in `nstat` during timeout investigations.

10. Performance Impact

Each demux/validation step is cheap (~ns), but lock contention on a hot listen socket historically limited accept rates → `SO_REUSEPORT` shards the listener across cores (used by NGINX, Envoy, Cloudflare).

11. Common Interview Questions

1. “A client gets connection timeouts but the server has low CPU. Where do you look?”
2. “What happens if a TCP segment arrives for a port nobody listens on?” (RST; for UDP: ICMP port unreachable.)
3. “Where can packets be dropped between the NIC and your application?”

12. Follow-up Questions

- “Difference between SYN queue and accept queue?” → SYN queue holds half-open; accept queue holds completed handshakes awaiting `accept()` .
- “What does `SO_REUSEPORT` solve?”

13. Debugging Scenarios

- `nstat | grep -i listen` shows overflows → app not accepting fast enough (GC pause? thread starvation?).
- `ethtool -S eth0 | grep drop` rising → ring buffer too small for burst rate.

14. Best Practices

- Monitor kernel drop counters as first-class metrics, not just app metrics.
- Set listen backlog explicitly (many frameworks default to 128).

15. Practice Questions

1. Java service with 2s GC pauses: explain the exact kernel mechanism by which clients see connect timeouts during the pause.

2. UDP receive buffer overflows show where? (`netstat -su` → “receive buffer errors”; per-socket `ss -u -m .`)

Topic 1.5 — MTU (Maximum Transmission Unit)

1. Why Interviewers Ask This

MTU bugs cause the most mystifying production failures (“small requests work, big ones hang”). Interviewers use it to test whether you’ve debugged real networks. Cloudflare and AWS interviews ask it directly.

2. Core Concept

MTU = largest IP packet a link can carry. Ethernet default: **1500 bytes**. Datacenter jumbo frames: **9000**. Anything bigger must be fragmented (IPv4, if allowed) or dropped with ICMP “Fragmentation Needed” (if DF bit set — which TCP sets by default for PMTUD).

3. Internal Working

Path MTU Discovery (PMTUD): sender sends full-size packets with DF=1. A router with a smaller egress MTU drops the packet and returns ICMP Type 3 Code 4 containing the supported MTU. Sender caches per-destination PMTU and resends smaller. **If a firewall blocks that ICMP → PMTUD blackhole**: big packets vanish forever, small ones sail through.

4. Packet Flow Explanation

1. Client sends 1500B packet, DF set.
2. VPN hop has MTU 1420 → drops packet, sends ICMP “frag needed, MTU 1420.”
3. Client TCP lowers effective MSS, retransmits at 1420.
4. If ICMP is filtered: client retransmits 1500B repeatedly → hang → eventual timeout. TCP’s fallback, **PLPMTUD (RFC 4821)**, probes with smaller packets after repeated losses — Linux enables it via `tcp_mtu_probing`.

5. ASCII Diagram

```
Client --1500B DF-->[R1 mtu1500]--X [R2 mtu1420]
                        |
                        <--ICMP frag-needed(1420)-- (if allowed)
Client --1420B DF-->[R1]-->[R2]--> OK

Blackhole case: ICMP filtered => client retransmits 1500B forever
Symptom: TLS handshake OK (small pkts), first big response hangs
```

6. Real Production Example

Widely-seen pattern: service works fine until it moves behind a VPN/overlay; then `git clone` and large POSTs hang while `curl` of small pages works. Root cause in postmortems at many companies: security team filtered *all* ICMP, killing PMTUD.

7. Advantages (of larger MTU / jumbo frames)

- 9000B frames: ~6x fewer packets per GB → less per-packet CPU, higher throughput for storage/replication traffic.

8. Trade-offs

- Jumbo frames must be consistent end-to-end within the L2 domain; mismatches cause silent loss.
- Larger packets = larger serialization delay and worse loss granularity (one loss burns 9KB).
- Internet paths are stuck at ≤ 1500 (often ~1400 with tunnels); you can't control external MTU.

9. Common Mistakes

- Blocking all ICMP “for security” — breaks PMTUD.
- Setting interface MTU 9000 while switches remain at 1500.
- Confusing MTU (L3 packet size) with MSS (TCP payload size).

10. Performance Impact

At 10 Gbps with 1500B packets $\approx$ 830k pps; with 9000B $\approx$ 140k pps. Per-packet costs (interrupts, lookups) scale with pps, so jumbo frames meaningfully cut CPU on storage/backup networks.

11. Common Interview Questions

1. “Small API calls succeed, large uploads hang. Diagnose.” (The canonical MTU question.)
2. “What is PMTUD and how does it fail?”
3. “Why is IPv4 fragmentation considered harmful?” (Loss of any fragment kills the whole packet; reassembly DoS; routers fragmenting costs CPU. IPv6 removed in-network fragmentation entirely.)

12. Follow-up Questions

- “How does TCP avoid fragmentation without ICMP?” → PLPMTUD probing.
- “What’s MSS clamping and where do you apply it?” → routers/tunnel endpoints rewrite the MSS option in SYNs.

13. Debugging Scenarios

- Test path MTU: `ping -M do -s 1472 host` (1472+28=1500). Shrink until it passes.
- `tracert` shows per-hop MTU.
- tcpdump signature of blackhole: repeated retransmits of the same full-size segment, no ACK progress.

14. Best Practices

- Allow ICMP Type 3 Code 4 (and IPv6 Packet Too Big) through every firewall you own.
- Enable `net.ipv4.tcp_mtu_probing=1` on internet-facing fleets.
- Clamp MSS at tunnel ingress.

15. Practice Questions

1. WireGuard (60B overhead) over PPPoE (8B) over 1500 Ethernet: compute safe tunnel MTU. (1500–8–60 = 1432.)
2. Why does the problem often appear *after* the TLS handshake succeeds? (Handshake packets are small; first data packet is full-size.)

Topic 1.6 — MSS (Maximum Segment Size)

1. Why Interviewers Ask This

MSS is the TCP-level twin of MTU and the mechanism behind MSS clamping — a standard fix interviewers expect you to know. It also leads into throughput math questions.

2. Core Concept

MSS = maximum TCP **payload** per segment. $MSS = MTU - IP\ header\ (20) - TCP\ header\ (20) \rightarrow 1460$ on standard Ethernet (less with TCP options like timestamps: effectively 1448 on Linux).

3. Internal Working

Each side advertises its MSS in the SYN (it is *not* negotiated — each direction uses the peer's advertised value, capped by its own PMTU estimate). MSS is an option only in SYN/SYN-ACK; it can't change mid-connection, but the effective segment size can shrink if PMTUD learns a smaller path MTU.

4. Packet Flow Explanation

1. Client SYN: `MSS=1460`.
2. Server SYN-ACK: `MSS=1460`.
3. Sender segments the byte stream into ≤ 1460 B chunks (or lets the NIC do it via TSO).
4. A middlebox doing MSS clamping rewrites the SYN option to e.g. 1360 so that resulting packets fit its tunnel — avoiding PMTUD entirely.

5. ASCII Diagram

```
SYN [MSS=1460] ---> [tunnel router: clamp to 1360] ---> server
SYN-ACK [MSS=1460] <-- clamp -- server
Result: both sides send <=1360B payload
       1360+20+20=1400 fits tunnel MTU of 1400. No ICMP needed.
```

6. Real Production Example

Virtually every home router and every cloud VPN gateway (AWS VPN, Tailscale, WireGuard guides) does MSS clamping — it's the pragmatic industry answer to ICMP-hostile networks.

7. Advantages

- Prevents fragmentation proactively, per-connection, without relying on ICMP.
- Zero cost after the handshake.

8. Trade-offs

- Clamping is a middlebox rewriting your packets (violates end-to-end); breaks if traffic is encrypted at L3 (IPsec — can't see the TCP header).
- Doesn't help UDP/QUIC — QUIC solves it with its own DPLPMTUD and a 1200B minimum datagram.

9. Common Mistakes

- Saying MSS is “negotiated to the minimum” — each side independently uses the other's advertisement (usually results in min, but the mechanism matters).
- Forgetting TCP options (timestamps = 12B) reduce real payload below MSS.
- Confusing MSS (payload) with MTU (whole IP packet).

10. Performance Impact

Throughput $\approx \text{cwnd} / \text{RTT}$, and cwnd is counted in MSS-sized segments. Initial cwnd = $10 \times \text{MSS} \approx 14.6\text{KB}$ — this is why keeping critical responses (e.g., TLS certs, initial HTML) under $\sim 14\text{KB}$ saves an RTT. A famous web-performance rule directly derived from MSS.

11. Common Interview Questions

1. “MTU vs MSS?”
2. “How does MSS clamping work and when do you need it?”
3. “Why do CDNs try to fit the critical response in 14KB?”

12. Follow-up Questions

- “What happens if you advertise MSS 9000 but the path is 1500?” → PMTUD/PLPMTUD shrinks effective size; if broken, blackhole.
- “How does TSO interact with MSS?” → kernel hands NIC a 64KB blob + MSS; NIC cuts segments.

13. Debugging Scenarios

- Read negotiated MSS live: `ss -tnpi | grep mss`.
- tcpdump SYN packets to verify clamping is applied: `tcpdump 'tcp[tcpflags] & tcp-syn != 0' -v` and check `mss` option.

14. Best Practices

- Clamp MSS at every tunnel/VPN boundary you operate (`iptables -j TCPMSS --clamp-mss-to-pmtu`).
- Keep latency-critical first responses within initial cwnd ($\sim 14\text{KB}$).

15. Practice Questions

1. MTU 1500 + TCP timestamps: what's the real max payload per segment? ($1500 - 20 - 20 - 12 = 1448$.)
2. RTT 100ms, loss forces cwnd ≈ 40 segments of MSS 1448. Max throughput? ($40 \times 1448\text{B} / 0.1\text{s} \approx 4.6\text{Mbps}$.)

MODULE 1 — One-Page Cheat Sheet

LAYERS THAT MATTER	L7 HTTP/gRPC/DNS L4 TCP/UDP L3 IP/ICMP L2 Eth/ARP
DEBUG ORDER	connectivity: bottom-up latency: top-down
KEY NUMBERS	Eth MTU 1500 jumbo 9000 MSS 1460 (1448 w/ ts) IP hdr 20B TCP hdr 20-60B VXLAN ~50B init cwnd 10*MSS≈14KB
ENCAPSULATION	each layer prepends header; tunnels nest whole packets
DECAPSULATION	drop points: NIC FCS, ring buffer, conntrack, listen backlog (SYN q + accept q), recv buffer (win=0)
PMTUD	DF bit + ICMP type3/code4; blocked ICMP => blackhole symptom: small OK, big hangs. Fix: allow ICMP, MSS clamp, tcp_mtu_probing=1
TOOLS	ss -tnpi nstat ethtool -S ping -M do -s N tracepath
CLOUD MAPPING	VPC = overlay encapsulation; K8s VXLAN eats 50B of MTU

MODULE 1 — Top Interview Questions

1. Small requests work, large ones hang — root cause and 3 fixes. (PMTUD blackhole → allow ICMP, MSS clamp, PLPMTUD.)
2. Walk a packet from `write()` to the wire and back up on the receiver.
3. L4 vs L7 device costs — why is deeper parsing slower?
4. How do pods with private IPs talk across nodes/VPCs? (Encapsulation/overlays.)
5. Where can the kernel silently drop inbound packets? Name 4 spots and their counters.
6. MTU vs MSS vs MRU; compute MSS for a given tunnel stack.
7. Why is IPv4 fragmentation avoided, and what did IPv6 change?

MODULE 1 — Common Mistakes

- Reciting 7 OSI layers instead of using them to localize a fault.
- “`write()` returned so the data was delivered.”
- Blocking all ICMP; forgetting Packet-Too-Big is load-bearing.
- Ignoring tunnel overhead in MTU math; mismatched jumbo frames.
- MSS “negotiation” misconception; forgetting options overhead.
- Trusting app logs over kernel drop counters.

MODULE 1 — Mock Interview (10 min)

Q1. “Your service migrated behind a new VPN. Health checks pass, small GETs pass, file uploads time out. Go.” *Strong answer:* immediately hypothesize PMTUD blackhole; verify with `ping -M do -s 1472` (fails) vs `-s 1300` (works); confirm filtered ICMP; fix via MSS clamping at the VPN gateway + allow ICMP 3/4; mention `tcp_mtu_probing` as a host-side mitigation.

Q2. “During traffic spikes, clients see connect timeouts; server CPU is 40%. Explain the kernel path that causes this.” *Strong answer:* SYN arrives → SYN queue → handshake completes → accept queue; if app

`accept()` is slow (GC pause, thread starvation) the accept queue overflows and SYN/ACKs are dropped silently; show `nstat` ListenOverflows; fix backlog + app accept loop.

Q3. Follow-up: “Would UDP have the same problem?” *Strong answer:* no handshake queues, but the UDP socket receive buffer overflows instead — drops visible in `netstat -su`; the application must handle loss.